



Computer Engineering Department
Computer Engineering
IC6400 - Operations Research

Project 0

Dr. Francisco J. Torres-Rojas

Dayron Josué Porras-Pérez
2023124693

Javier Lee-Liang
2020184824

José Julián Araya-Castillo
2020098909

San José, Costa Rica
September 2026

Experiment Mode

Conditions

This file was generated to see the results of three different algorithms solving multiple 100 random generated 0/1 Knapsack Problem and store the diverse results everytime the algorithm solves each problem. The experiment done has specific conditions everytime the **knapsack** program is run.

- The experiment tests 100 different 0/1 Knapsack Problems.
- When running the experiment mode with the **knapsack** program, it needs to receive a parameter of how many iterations to do. This is the parameter **-E=n**, where **n** indicates the amount of iterations to do the experiment.
- Each problem will be solved by three algorithms and data will be collected and stored in tables with the same format as shown in Table 1.
- Each cell represents a specific problem. The rows indicate the size of the knapsack for the the problem and the columns indicate how many items are in the problem.
- Each problem also has specific conditions, because each iteration of the table generates a random problem:
 - Value of items are in the range of 0 and 100.
 - Cost of items are in the range of 0 and 40% of the knapsack capacity.
- When running the program with the amount of iterations, it will generate 100 random problems with the conditions explained before, it will solve each other, store data and it will do other 100 random problems until the amount of iterations are satisfy.
- The data collected for the tables will be an overall average or percentage of all iterations done.
- At the end the experiment returns 5 tables.
 1. Table of average times needed to solve each problem with the Dynamic Programming Algorithm.

2. Table of average times needed to solve each problem with the Basic Greedy Algorithm.
3. Table of average times needed to solve each problem with the Proportional Greedy Algorithm.
4. Table of percentages of how often the Basic Greedy Algorithm finds the optimal solution of a problem.
5. Table of percentages of how often the Proportional Greedy Algorithm finds the optimal solution of a problem.

	Amount of items									
Knapsack	10	20	30	40	50	60	70	80	90	100
100										
200										
300										
400										
500										
600										
700										
800										
900										
1000										

Table 1: Format of the tables

Algorithms

Dynamic Programming Algorithm

This algorithm uses a table to get to the optimal solution for the generated 0/1 Knapsack Problem. The strategy here is to solve subproblems, write down the optimal solution for that subproblem on a table and by using those solutions build the solution for the original problem. It always find the optimal solution for a 0/1 Knapsack Problem, as explained in course of Operations Research.

Basic Greedy Algorithm

This algorithm focuses on trying to put inside the knapsack the most valuable item, if it does not fit in, tries with the next most valueable item until the are no more items available.

Proportional Greedy Algorithm

It has the same strategy as the Basic Greedy, but it focuses on trying to put inside items based by their proportion between its value and cost. The item with highest proportion goes first, if it does not fit, goes to the next item with the highest proportion.

Experiment Results

Average execution times in seconds (4 decimal places)

Dynamic Programming

This is the table of times used by the Dynamic Programming Algorithm.

	Amount of items									
Knapsack	10	20	30	40	50	60	70	80	90	100
100	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan
200	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan
300	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan
400	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan
500	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan
600	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan
700	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan
800	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan
900	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan
1000	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan

Basic Greedy

This is the table of times used by the Basic Greedy Algorithm.

	Amount of items									
Knapsack	10	20	30	40	50	60	70	80	90	100
100	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan
200	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan
300	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan
400	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan
500	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan
600	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan
700	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan
800	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan
900	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan
1000	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan

Proportional Greedy

This is the table of times used by the Proportional Greedy Algorithm.

	Amount of items									
Knapsack	10	20	30	40	50	60	70	80	90	100
100	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan
200	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan
300	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan
400	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan
500	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan
600	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan
700	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan
800	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan
900	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan
1000	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan	-nan

Conlusions of the experiment

As shown on the tables before, the Dynamic Programming Algorithm seems to take more time, because it searches the optimal solution for every subproblem of the original problem. Compared to the Greedy Algorithms they take less time, because it just sorts the items by value, for the Basic Greedy, or by proportion, for the Proportional Greedy, and just looks for the items that can fit in the sorted list. The times between these two are not significant enough.

Collisions percentage

Dynamic Programming

There is no need to create a table for this algorithm. This algorithm is used to give us the optimal solutions for each problem in the table.

Basic Greedy

This table shows the percentage of collisions of the Basic Greedy Algorithm compared to the optimal solution.

	Amount of items									
Knapsack	10	20	30	40	50	60	70	80	90	100
100	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%
200	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%
300	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%
400	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%
500	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%
600	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%
700	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%
800	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%
900	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%
1000	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%

Proportional Greedy

This table shows the percentage of collisions of the Proportional Greedy Algorithm compared to the optimal solution.

	Amount of items									
Knapsack	10	20	30	40	50	60	70	80	90	100
100	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%
200	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%
300	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%
400	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%
500	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%
600	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%
700	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%
800	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%
900	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%
1000	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%	-nan%

Conclusions of the experiment

By executing the three algorithms and comparing the results of the Dynamic Programming Algorithm between the two Greedy Algorithms, the results show that the Basic Greedy Algorithm does not hit the same optimal solutions as the Proportional Greedy Algorithm. But a pattern can be noted, the algorithms do not scale well when amount of items increase. By increasing the size of the knapsack, looks like both algorithms tend to hit less optimal solutions, but not as much by increasing the amount of items.